Creating and Using Python Virtual Environments (venv) Effectively

Quick practical guide for reproducible projects • February 08, 2026

Why venv matters: A virtual environment isolates your project's dependencies (and their versions) from other projects and from your system Python. That isolation is the foundation of reproducibility: your code runs the same way on another machine when recreated from a requirements file.

Key idea: A venv is just a folder containing a Python interpreter + site-packages. Activate it to make your shell use that interpreter by default.

1) Create a virtual environment

Choose one folder per project, then create a **.venv** inside that folder. The name can be anything; **.venv** is a common convention.

Windows (PowerShell or CMD)

```
cd your_project_folder
py -m venv .venv
```

macOS / Linux (bash/zsh)

```
cd your_project_folder
python3 -m venv .venv
```

If you have multiple Python versions installed, venv uses the interpreter you run it with. That's why using **py** (Windows launcher) or **python3** (macOS/Linux) matters.

2) Activate (and verify) the environment

Activation updates your shell so that **python** and **pip** refer to the venv's interpreter and installer.

Windows

```
.venv\Scripts\activate
```

macOS / Linux

```
source .venv/bin/activate
```

Verify you are using the venv interpreter

```
python -c "import sys; print(sys.executable)"
python -V
pip -V
```

Expected: the printed path points inside **.venv**. If it does not, you are not activated (or your shell is using a different interpreter).

3) Install packages the right way

Always use **python -m pip** to ensure pip matches the interpreter you intend to use.

```
python -m pip install --upgrade pip
python -m pip install pandas numpy matplotlib
```

Quick check:

```
python -c "import pandas, numpy, matplotlib; print('OK')"
```

4) Create and use requirements.txt

A requirements file lets someone recreate your environment. There are two common approaches:

A) Snapshot (fast): freeze exact installed versions (good for course labs and small projects).

```
python -m pip freeze > requirements.txt
```

B) Minimal + lock (more robust): keep a minimal list (human-edited) and generate a lock file using tools like pip-tools/poetry/uv. This is better for teams and long-lived codebases.

Recreate the environment from requirements.txt

```
# (1) create venv
py -m venv .venv   # or: python3 -m venv .venv
# (2) activate
.venv\Scripts\activate  # Windows
# source .venv/bin/activate  # macOS/Linux
# (3) install
python -m pip install -r requirements.txt
```

5) Best practices that prevent 80% of problems

- **One venv per project.** Do not reuse one environment across unrelated projects.
- **Do not commit .venv/ to Git.** Commit **requirements.txt** (and optionally **pyproject.toml**).
- **Pin critical versions** when results matter (e.g., ML coursework). Example: *numpy==2.0.1*.
- **Keep commands repeatable.** Put setup steps in a short README: create venv → activate → install → run.
- **Prefer python -m pip** over plain pip to avoid interpreter mismatch.

Command cheat sheet

Action	Windows	macOS/Linux
Create venv	py -m venv .venv	python3 -m venv .venv
Activate	.venv\Scripts\activate	source .venv/bin/activate
Deactivate	deactivate	deactivate
Install package	python -m pip install <pkg>	python -m pip install <pkg>
Export requirements	python -m pip freeze > requirements.txt	python -m pip freeze > requirements.txt
Install from requirements	python -m pip install -r requirements.txt	python -m pip install -r requirements.txt

6) Troubleshooting (quick fixes)

Activation doesn't work (Windows PowerShell policy). Run: **powershell -ExecutionPolicy Bypass -File .venv\Scripts\Activate.ps1**

pip installs but your code can't import the package. You likely installed into a different interpreter. Use **python -m pip install ...** and re-check **sys.executable**.

Your venv is huge or slow to install. Use fewer heavy packages, or consider a lockfile workflow (pip-tools/poetry/uv). For class labs, keep dependencies minimal.

turtle or Tkinter issues on some systems. turtle is standard-library; if it fails, your Python build may be missing Tk support. Install a standard Python distribution that includes Tk.

7) A simple reproducible workflow you can copy-paste

```
# 1) Create project folder and venv
mkdir my_project
cd my_project
py -m venv .venv
.venv\Scripts\activate

# 2) Install deps
python -m pip install --upgrade pip
python -m pip install pandas numpy matplotlib

# 3) Freeze deps
python -m pip freeze > requirements.txt

# 4) Run your program
python main.py
```